
LAPORAN PRAKTIKUM INTERNET OF THINGS



Disusun oleh :

Nama : Restu anggara

NPM : 25782051

Kelas : TRI-3B

Dosen :

Dr.Ir.Septafiansyah Dwi Putra,S.T.,M.T.

Agus Ambarwari, S.Pd.,M.Kom.

Andi Saptono,S.Kom

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG**

2026/2027

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

The screenshot shows the Arduino IDE environment. The top menu bar includes File, Edit, Sketch, Tools, Help, and a toolbar with icons for saving, undo, redo, and running. The main window displays a C++ sketch named "sketch_sep21a.ino". The code defines a NodeMCU ESP-12E module with various pins and variables.

```

1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "cihuuyy";
7 const char* password = "12345678";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2;      // D4 (GPIO 2)
11 const byte buttonPin = 4;   // D2 (GPIO 4)
12 const byte ledPin = 12;     // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT22);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";

```

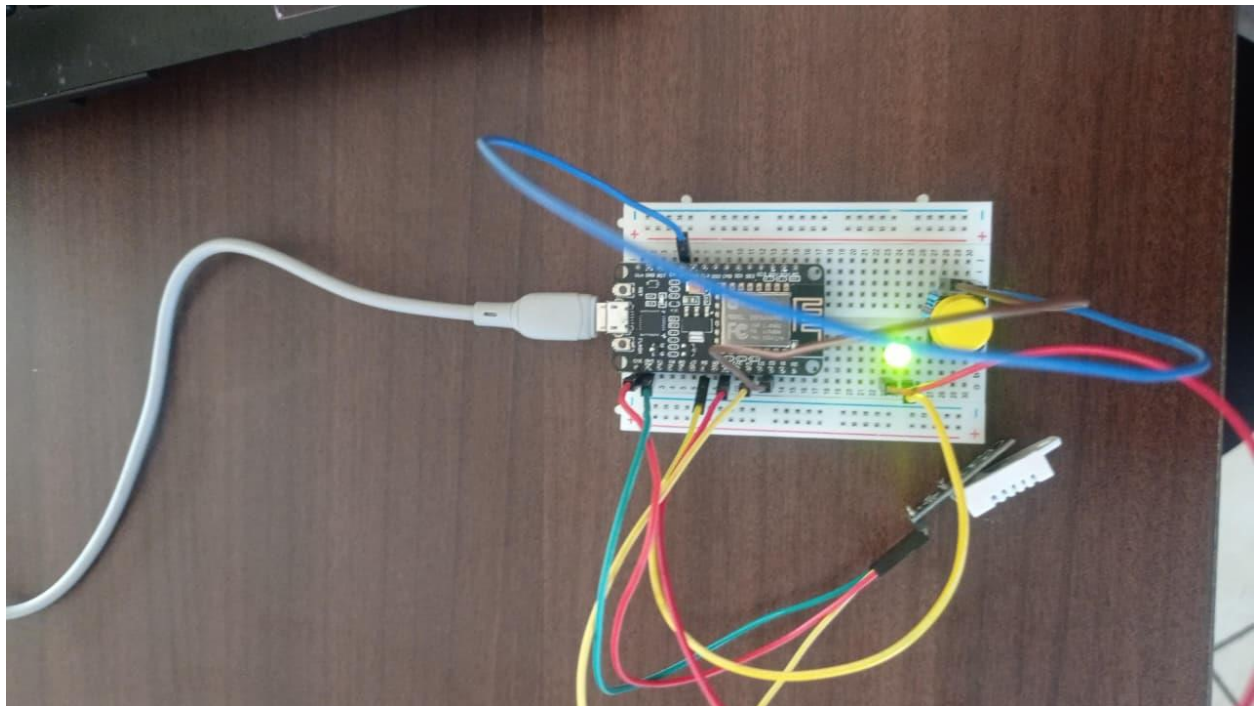
Below the sketch editor is the Serial Monitor window, which is open at 115200 baud. It shows a series of messages from the client WebSocket #1, indicating successful connections and disconnections.

```

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line 115200 baud

14:29:30.634 -> Client WebSocket #1 terhubung
14:30:00.502 -> IP Address: 10.58.129.30
14:30:07.945 -> IP Address: 10.58.129.30
14:30:13.835 -> Client WebSocket #1 terhubung
14:32:16.616 -> Client WebSocket #1 terputus
14:32:16.745 -> Client WebSocket #2 terhubung
14:32:19.182 -> Client WebSocket #2 terputus
14:32:19.316 -> Client WebSocket #3 terhubung

```



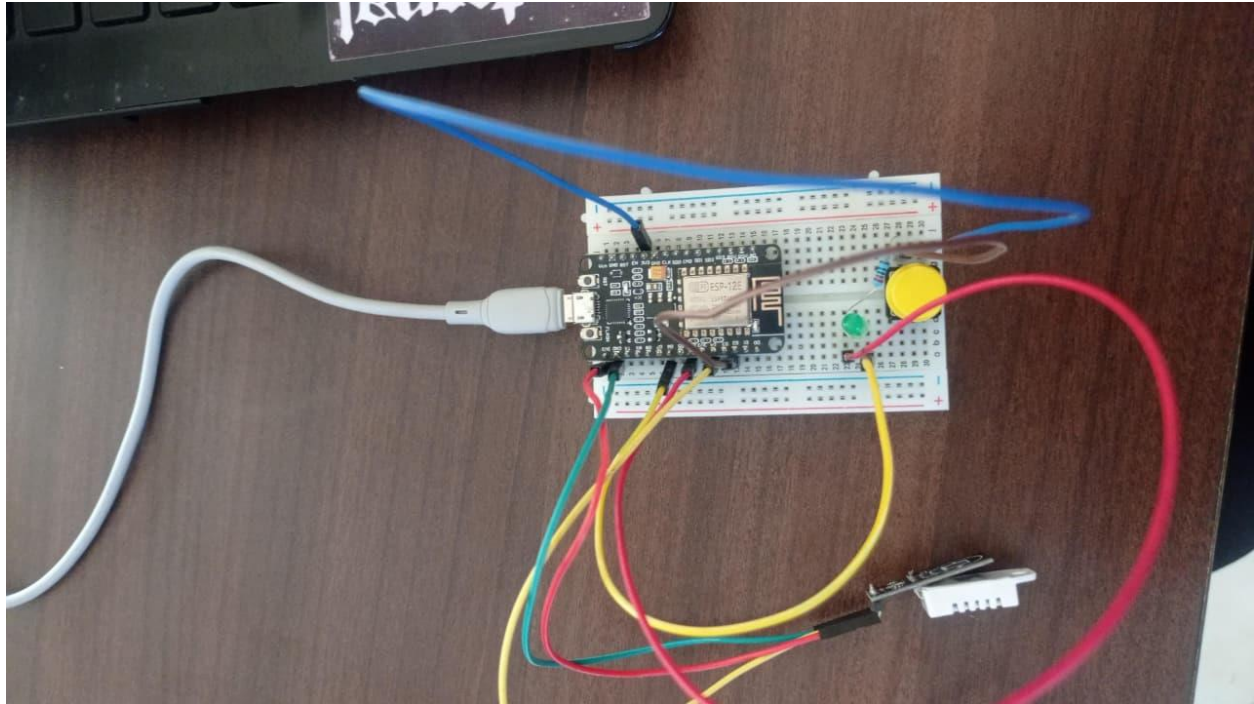
A screenshot of a web browser's address bar. The address bar shows a warning icon (a triangle with an exclamation mark) and the text "Not secure". To the right of this, the IP address "10.58.129.30" is displayed. On the far right of the address bar, there are several icons: a document icon, a star icon, a download icon, a globe icon, and a menu icon (three vertical dots). Below the address bar, a portion of the browser's tab bar is visible, showing icons for Gmail, YouTube, and Maps, followed by a tab labeled "All Bookmarks".

Smart Room

Suhu: 67.30 Celcius

LED: ON

Turn OFF



Smart Room

Suhu: 67.30 Celcius

LED: OFF

Turn ON

Langkah Kerja Rangkaian:

1. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND** NodeMCU (*sama seperti rangkaian LED pada Modul 3*).
2. Hubungkan pin **DATA / OUT** DHT ke pin **D4 (GPIO 2)**. (VCC DHT ke 3V3, GND ke GND).
3. Pasang **Push Button** pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur **3V3 (3.3V)**. Hubungkan kaki seberang tombol ke pin **D2 (GPIO 4)** NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki **Resistor 10k Ohm**, lalu hubungkan ujung kaki resistor lainnya ke jalur **GND** (*konfigurasi resistor pull-down eksternal seperti pada Modul 1*).

4. Buka IP Address tersebut di browser laptop Anda, dan sekaligus buka IP yang sama di browser ponsel (smartphone) Anda secara berdampingan.
5. Uji 1 (Kendali Web Tersinkronisasi): Klik tombol pada browser laptop Anda. Perhatikan bagaimana browser di ponsel Anda ikut berubah (Turn ON) secara presisi, dan lampu LED fisik ikut menyala!
6. Uji 2 (Kendali Perangkat Keras): Sentuh push button mekanik di meja Anda. Amati bahwa antarmuka browser di laptop maupun ponsel Anda secara serentak berganti status (Turn OFF menjadi Turn ON) tanpa terjadi proses loading atau refresh HTML halaman sama sekali! Kecepatan respon seketika ini adalah manfaat dari pemrograman Asynchronous dan pemisahan (decoupling) pembacaan memori sensor DHT.

Kode perintah:

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

// Konfigurasi Pin
const byte dhtPin = 2;           // D4 (GPIO 2)
const byte buttonPin = 4;        // D2 (GPIO 4)
const byte ledPin = 12;          // D6 (GPIO 12)

DHT dht(dhtPin, DHT11);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;

// Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ----- HTML & JAVASCRIPT (FRONT-END) -----
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Real-Time IoT Web</title>
  <style>
    body { font-family: Arial; text-align: center; }
```

```

    .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-width: 300px;
border-radius: 10px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 5px; cursor: pointer;
color: white;}
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
</style>
</head>
<body>
    <h1>Smart Room</h1>
    <div class="card">
        <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
    </div>
    <div class="card">
        <h2>LED: <span id="ledStatus">OFF</span></h2>
        <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
    </div>

<script>
    // Membuka terowongan koneksi WebSocket ke alamat IP ESP8266
    var gateway = `ws://${window.location.hostname}/ws`;
    var websocket;

    // Menjalankan inisiasi saat halaman pertama kali dimuat
    window.addEventListener('load', onLoad);
    function onLoad(event) { initWebSocket(); }

    function initWebSocket() {
        websocket = new WebSocket(gateway);
        websocket.onopen    = onOpen;
        websocket.onclose   = onClose;
        websocket.onmessage = onMessage;
    }

    function onOpen(event) { console.log('WebSocket Terkoneksi'); }
    function onClose(event) { setTimeout(initWebSocket, 2000); }

    // Dipanggil saat tombol di layar web ditekan
    function toggleLed(){
        websocket.send('toggle');
    }

    // Menangkap paket data JSON yang "didorong" (PUSH) oleh C++ NodeMCU
    function onMessage(event) {
        var dataObj = JSON.parse(event.data);

        // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
        if(dataObj.suhu !== undefined) {
            document.getElementById('tempValue').innerHTML = dataObj.suhu;
        }

        // Merombak UI Tombol & Teks sesuai status hardware
        if(dataObj.led !== undefined) {
            var btn = document.getElementById('toggleBtn');
            var status = document.getElementById('ledStatus');

```

```

        if(dataObj.led == "1"){
            status.innerHTML = "ON";
            btn.innerHTML = "Turn OFF";
            btn.className = "btn-on";
        } else {
            status.innerHTML = "OFF";
            btn.innerHTML = "Turn ON";
            btn.className = "btn-off";
        }
    }
}
</script>
</body>
</html>
)rawliteral";

// ----- BACK-END & WEBSOCKET LOGIC -----

// Fungsi menyebarkan paket data JSON ke SELURUH browser pengunjung
void notifyClients() {
    String jsonString = "{\"led\": \"" + String(ledState ? 1 : 0) + "\", ";
    jsonString += "\"suhu\": \"" + currentTemp + "\"}";
    ws.textAll(jsonString);
}

// Handler pesan masuk (dari klik browser)
void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
    AwsFrameInfo *info = (AwsFrameInfo*)arg;
    if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT)
    {
        data[len] = 0;
        if (strcmp((char*)data, "toggle") == 0) {
            ledState = !ledState;
            notifyClients();
        }
    }
}

// Event handler WebSocket bawaan AsyncWebServer
void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type,
             void *arg, uint8_t *data, size_t len) {
    switch (type) {
        case WS_EVT_CONNECT:
            Serial.printf("Client WebSocket #%u terhubung\n", client->id());
            notifyClients(); // Kirim status terkini kepada klien yang baru buka web
            break;
        case WS_EVT_DISCONNECT:
            Serial.printf("Client WebSocket #%u terputus\n", client->id());
            break;
        case WS_EVT_DATA:
            handleWebSocketMessage(arg, data, len);
            break;
    }
}

```

```

void setup() {
    Serial.begin(115200);

    // Mengatur pin tombol sebagai input (dengan resistor pull-down eksternal)
    pinMode(buttonPin, INPUT);

    pinMode(ledPin, OUTPUT);
    digitalWrite(ledPin, LOW);
    dht.begin();

    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.println("\nIP Address: " + WiFi.localIP().toString());

    server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
        request->send_P(200, "text/html", index_html);
    });

    // Ikat handler WebSockets ke server utama
    ws.onEvent(onEvent);
    server.addHandler(&ws);
    server.begin();
}

void loop() {
    ws.cleanupClients();

    // 1. Eksekusi perangkat keras LED
    digitalWrite(ledPin, ledState ? HIGH : LOW);

    // 2. Baca Tombol Fisik (Debounce) tanpa nge-freeze program
    int reading = digitalRead(buttonPin);
    if (reading != lastButtonState) {
        lastDebounceTime = millis();
    }
    if ((millis() - lastDebounceTime) > debounceDelay) {
        if (reading != buttonState) {
            buttonState = reading;

            // Karena menggunakan resistor PULL-DOWN eksternal, tombol ditekan = HIGH
            if (buttonState == HIGH) {
                ledState = !ledState;
                notifyClients(); // PUSH DATA KE BROWSER SEKETIKA
            }
        }
    }
    lastButtonState = reading;

    // 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
    if ((millis() - lastTime) > 3000) {
        float t = dht.readTemperature();
        if (!isnan(t)) {
            currentTemp = String(t);
            notifyClients(); // Push pembaruan suhu otomatis
        }
    }
}

```

```
}  
  lastTime = millis();  
}  
}
```

Penjelasan Singkat Kode Khusus:

- `ws.cleanupClients()`: (Berada di siklus `loop()`). Fungsi penting dari pustaka *Asynchronous* ini bertugas membersihkan dan menutup sesi (*resource memori*) dari klien/*browser* yang sudah terputus atau menutup tab layarnya. Tanpa baris ini, memori RAM ESP8266 akan penuh (*memory leak*) dan berujung pada *Crash/Restart* jika terlalu banyak perangkat keluar-masuk membuka halaman *Dashboard*.
- `pinMode(buttonPin, INPUT)` dan `if (buttonState == HIGH)`: Mempertahankan konfigurasi resistor *Pull-Down* eksternal 10k Ohm ke jalur GND (seperti pada Modul 1). Saat tombol dilepas, pin D2 membaca logika **LOW** (0V). Saat tombol ditekan, arus 3.3V mengalir sehingga terbaca logika **HIGH** yang memicu pembalikan status (*toggle*) LED secara instan.
- `if ((millis() - lastTime) > 3000)`: Pemrograman *non-blocking*. Alih-alih membuat CPU tidur selama 3 detik menggunakan `delay(3000)` (yang akan membekukan respons WebSocket), kita menggunakan selisih penghitung jam internal CPU (`millis()`). Dengan demikian, CPU dapat terus siaga mengawasi sentuhan tombol fisik dan interaksi jaringan internet dengan kecepatan tinggi.
- `var dataObj = JSON.parse(event.data)`: Sintaks JavaScript yang membedah teks kaku kiriman C++ berformat `{"suhu": "30", "led": "1"}` menjadi struktur data objek dinamis (*Dictionary*) untuk memperbarui elemen desain HTML dengan instan (*DOM Manipulation*).
- `ws.textAll()`: Perintah penyiaran (*broadcasting*) ampuh yang menembakkan/menyebarkan data secara seketika ke *seluruh* perangkat *browser* pengunjung (laptop, HP) secara bersamaan.

Kesimpulan:

Program ini menggunakan **ESP8266, DHT22, LED, tombol, dan WebSocket** untuk membuat sistem monitoring dan kontrol IoT secara real-time. Sensor DHT22 membaca suhu setiap 3 detik menggunakan `millis()` tanpa menghentikan program, sedangkan WebSocket mengirim perubahan data langsung ke browser tanpa melakukan refresh halaman. LED juga dapat dikontrol melalui tombol fisik maupun tombol pada web.

4.6 Latihan

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

WebSockets lebih hemat bandwidth karena koneksi tetap terbuka dan hanya mengirim data yang diperlukan. Berbeda dengan Meta Refresh yang memuat ulang seluruh halaman secara berkala, sehingga lebih banyak memakai data dan membuat layar berkedip.

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan *NodeMCU*) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Bahaya utamanya adalah komunikasi WebSockets menjadi lambat dan tidak responsif. `delay(3000)` menghentikan `loop()` selama 3 detik, sehingga *NodeMCU* terlambat menerima atau mengirim data dan respons browser bisa delay, terputus, atau timeout.

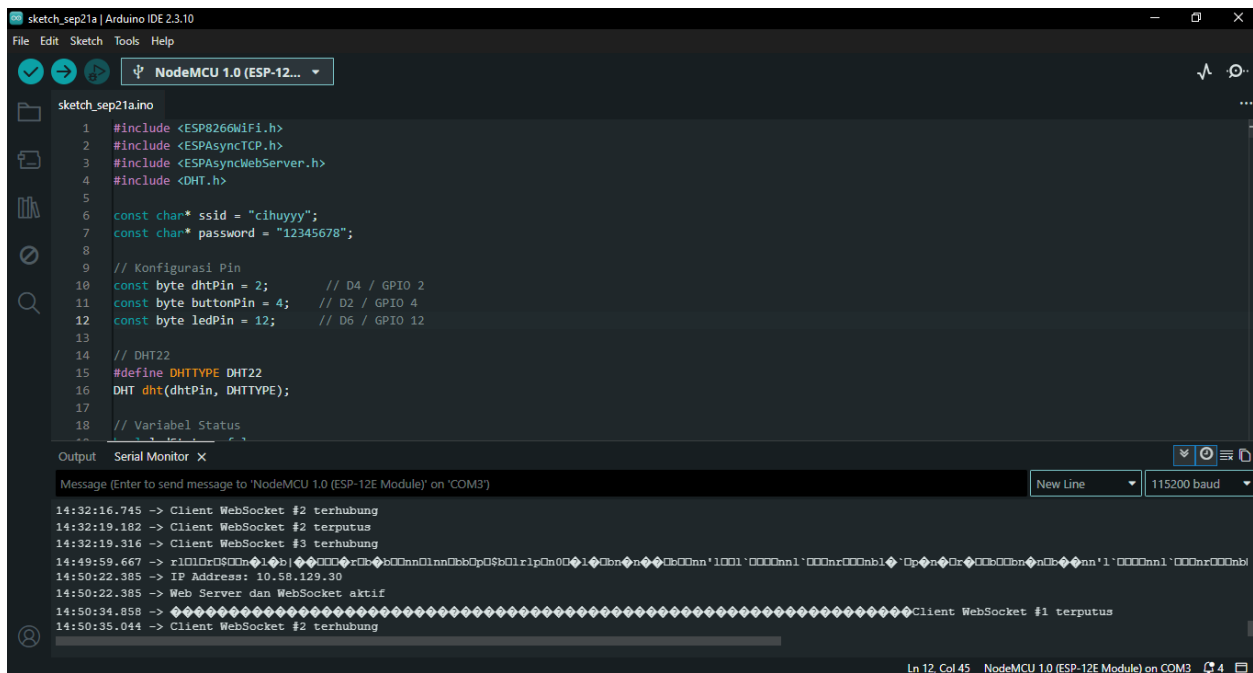
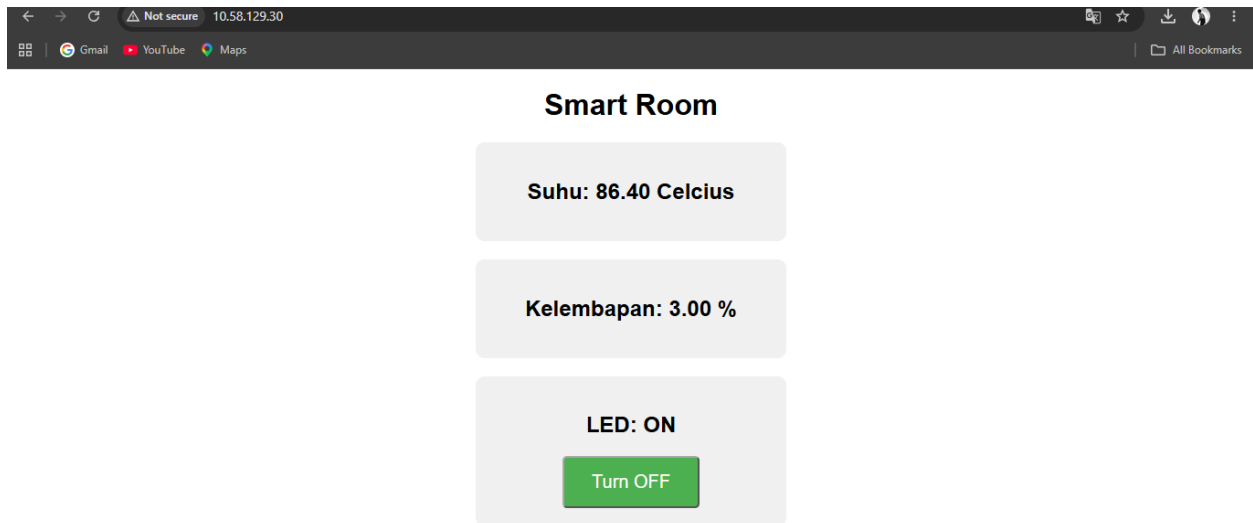
- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

ID elemen HTML yang menjadi target perubahan oleh JavaScript adalah:

- **tempValue** → mengubah tampilan nilai suhu.
- **ledStatus** → mengubah teks status LED menjadi **ON/OFF**.
- **toggleBtn** → mengubah teks tombol menjadi **Turn ON/Turn OFF**.

Jadi, ID yang ditargetkan adalah: **tempValue**, **ledStatus**, dan **toggleBtn**.

- **Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
 - i. **Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).



- ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (<div class="card">) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue. Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

1. C++ — Tambahkan variabel kelembapan

String currentHum = "--";

```
#define DHTTYPE DHT22  
DHT dht(dhtPin, DHTTYPE);
```

Pada timer millis():

```
float h = dht.readHumidity();
```

```
if (!isnan(t) && !isnan(h)) {  
    currentTemp = String(t);  
    currentHum = String(h);  
    notifyClients();  
}
```

Ubah notifyClients():

```
jsonString += "\"suhu\": \"" + currentTemp + "\", ";  
jsonString += "\"hum\": \"" + currentHum + "\"}";
```

2. HTML — Tambahkan kartu kelembapan

```
<div class="card">  
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>  
</div>
```

3. JavaScript — Tambahkan pada onMessage()

```
if(dataObj.hum !== undefined) {  
    document.getElementById('humValue').innerHTML = dataObj.hum;  
}
```

Jadi modifikasinya: C++ membaca dan mengirim data kelembapan, HTML menyediakan tempat tampilannya, dan JavaScript menampilkan data tersebut secara real-time.